

THE OMNI-COMPASS MANUAL

The Governor, Its Mechanism, and How to Wire It onto Your Stack

Edition 1.0, October 2026 The Omni-Compass LLC

PROPRIETARY - EVALUATION AND SIMULATION USE ONLY. Copyright (c) 2026 The Omni-Compass LLC. All rights reserved. SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0. This manual and the software it describes are not open source. You may use them only to evaluate Omni-Compass and to reproduce its published results, including in shadow or test mode on systems you own or control. Any commercial use, commercialization, monetization, production use, operation of any system beyond evaluation, redistribution, hosted or managed service, or incorporation into any product or service requires a written **Omni-Compass Enterprise License**, signed by The Omni-Compass LLC and paid for. Omni-Compass is protected by copyright and by patents and patent applications held by The Omni-Compass LLC. "Omni-Compass" and its marks are trademarks of The Omni-Compass LLC. Full terms: LICENSE and NOTICE.

FRONT MATTER

Notice and Disclaimer

The software is provided "as is", without warranty of any kind. Every result in this manual carries its evidence class (section 14). A simulated result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this manual is a promise of a particular saving on a particular system. The only number that applies to your system is the one your own paired runs produce, on your own receipt. Before Omni-Compass writes to any production system, it must run in watch mode, pass the wire check, and be covered by a signed Omni-Compass Enterprise License.

Foreword

I built Omni-Compass to sit on top of what already exists. Kubernetes, the GPU driver, the building controller, the battery inverter, the robot's servo loop: those are muscles, and they are good muscles. What none of them has is one brain that reads all of them at once, holds each of them in the middle of its safe range, and puts every setting back exactly where it found it when it stops. That brain is Omni-Compass.

This manual is the whole of it: what the governor is, the mathematics it runs on, how its nervous system reads and writes, how it is wired to each kind of machine, how to turn it on one level at a time, how to switch it off, and how to prove on your own system what it does. Keep it with the code. When the code changes, this manual changes in the same commit.

AJ Dubra Founder, The Omni-Compass LLC

Preface: How to Use This Manual

You are	Read
CEO, board member, investor	Foreword; Part I (sections 1-3); section 14 (evidence); section 16 (license); the Executive Summary below
CTO, architect, head of platform	Parts I-III; section 9 (wiring levels) to plan the rollout; Part VI
The engineer wiring it	Everything, in order. Do not skip the wire check (section 8) or the watch level (section 9, level 1)
Auditor, diligence team	Part II (mechanism), Part VI (proof), Appendix C (equations), Appendix F (evidence map)

Conventions. code is a command, file or switch exactly as typed. "Native" means your system as it runs today, without Omni-Compass. "Muscle" means any machine, service or controller Omni-Compass can read and set. "Knob" or "lever" means one setting on a muscle. "The band" is a knob's or a service reading's safe range. Every step that writes to a system is marked **WRITES**.

Executive Summary

- **What it is.** A supervisory governor. It reads the meters of every muscle in your stack, computes one bounded force per muscle from a single closed mathematical law, and moves each muscle's own setting (a replica target, a clock ceiling, a power limit, a setpoint) to hold the service in the middle of its band. The muscles keep their own controls. Omni-Compass sets what they already accept.
- **What it does for you.** It turns the room your systems keep "just in case" into either more work for the same energy or the same work on fewer machines and fewer watts. On real Kubernetes (two independent sets of ten paired runs) it served the same traffic with about a third fewer machines in service, responses about 60% faster at the 95th percentile, and zero failed requests (section 15).

- **How it stays safe.** It watches before it writes, records every original setting before it acts, reads back every write, never writes past a knob's cover, gives everything back the moment service is at risk, stops writing if anyone else touches a knob, and returns every setting to its original value on one OFF switch.
- **How you know.** Every run produces receipts: native against Omni-Compass on the same system, the same load and the same clock, with 95% intervals. Results are labelled by a rule written before the run.

Table of Contents

Front Matter - Notice and Disclaimer; Foreword; Preface; Executive Summary

Part I - The Governor

1. What Omni-Compass Is
2. The Muscles, the Realms and the Six Organisms
3. Where the Value Comes From

Part II - The Mechanism of Action

4. The Engine: Eight Equations and One Control Law
5. The Closed Circle: Why It Cannot Leave Its Bowl
6. The Bowl: Push, Pull and the Two Forces
7. The Two-Way Nervous System

Part III - The Harness: Plugs, Wires and the Wire Check

8. The Plug, the Adapters and the Wire Check

Part IV - Wiring It onto Your Stack, Step by Step

9. Before You Start, and the Eight Levels
10. Stack by Stack

Part V - Operating It

11. The OFF Switch, the Rules, and the Log
12. Maintenance, Upgrades and Security

Part VI - Proving It

13. Paired Runs and Receipts on Your Own System
14. Evidence Classes and How to Read a Result
15. Results to Date

Part VII - Code, Twins and Terms

16. License and Commercial Terms
17. Python, C++ and the Seal

Back Matter - Glossary; Appendix A Command Reference; Appendix B File Map; Appendix C The Equations in Full; Appendix D Metrics; Appendix E Troubleshooting; Appendix F Evidence Map; Contact

PART I - THE GOVERNOR

1. What Omni-Compass Is

A governor on a steam engine does not build the engine and does not turn the shaft. It watches the speed and moves the throttle so the speed stays in a band. Omni-Compass is that, for every machine you run.

It is a process on a host. It reads meters, steps a bounded mathematical law, and writes only the levers it has been given. It is not the chip, not the GPU driver, not Kubernetes, not the building controller. Those keep running exactly as they do today; Omni-Compass sets the values they already accept:

Muscle	Its own control (kept)	What Omni-Compass sets
Kubernetes service	the HPA	the HPA's CPU target; replica floors
Kubernetes node pool	Cluster Autoscaler / Karpenter / MachineSet	how many machines stay in service (park and wake)
NVIDIA GPU	the card's firmware (boost, power and thermal limits)	the clock ceiling (up wire) and the power limit (down wire)
CPU	the kernel's frequency governor	the frequency ceiling
Data hall, building	the chiller and air handler loops	the supply-air setpoint; units in service
Battery site	the inverter	the reserve level; the peak ceiling
Robot joint, vehicle axis	the servo loop	speed and effort limits
Process loop, feeder	the PI controller	the setpoint inside its band

When Omni-Compass stops, every one of those values goes back to what it was before Omni-Compass acted.

2. The Muscles, the Realms and the Six Organisms

The catalog (`realms/catalog.csv`) lists 656 distinct muscles, each with its family, its plant model and its knob. They fall into four realms. Every realm stands on the same spine (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution, 190 muscles), plus its own domain muscles.

Organism	Muscles	What it is
1. Compute / AI / Cloud	345	clusters, GPUs, AI training and inference, cloud capacity
2. Physics / Robotics / Autonomous	262	joints, fleets, vehicles, flight and spacecraft axes
3. Energy / Facility / Industrial	282	data halls, buildings, batteries, UPS, process loops, feeders
4. Distribution / Specialized	337	networks, storage, databases, commerce, workflows, radio networks
5. The four stacked, every duplicate kept	1,226	all four realms on one clock, the shared spine counted in each
6. The whole tower, every muscle once	656	every distinct muscle on one clock

These six organisms are the benchmark set. Each is run native and with Omni-Compass on top, on the same seed, the same load and the same clock, and each produces its own receipt.

3. Where the Value Comes From

Every system runs with room it does not use: GPUs boost to the top of their clock range and are knocked back by their own power limiter many times a second; Kubernetes keeps replicas and machines sized for the worst minute; cooling runs

colder than the heat requires; batteries hold more reserve than the hour needs. That room is paid for in energy and in machines.

Omni-Compass holds each service in the middle of its band instead of far below its limit. The room that was spent on nothing becomes one of two things, and the receipt shows which:

- **More work for the same energy** (work per energy rises), or
- **The same work for less** (fewer machines in service, fewer watts).

They are the same gain read from two sides. The receipt reports one number, work per energy, and beside it the machines, the response times and the failures, so nothing is hidden.

PART II - THE MECHANISM OF ACTION

4. The Engine: Eight Equations and One Control Law

The engine (omnicompass/core.py, frozen and fingerprinted) carries a six-part state $x = (E, U, I_U, S, B, B_{\dot{}})$:

State	Meaning	Physical picture
E	deviation: how far the system is from where it should be	a tank that fills with stress and drains on its own
U	alignment, between the two poles -1 and +1	a ball in a double well: two stable poles, a hill between them
I_U	memory of misalignment	an integrator that remembers and slowly forgets
S	basin structure	a ball rolling to the bottom of its landscape
B, $B_{\dot{}}$	the bath	a spring with friction that absorbs and settles energy

The eight equations (Appendix C gives them in full):

1. $dE/dt = -\alpha_E E + \beta_{int} + \beta_{ext} + v_{eff}$
2. $dU/dt = \mu U (1 - U^2) - (dE/dt)/E_{max} - \lambda_U U + u$, with $|u| \leq 25$
3. $dI_U/dt = (1 - U) - \sigma_1 E - \delta S - \lambda_I I_U$
4. $v_{eff} = \cos(\omega_B t / 2) c \tanh(\lambda_0 + \lambda_1 (U - 0.5) + \lambda_2 S)$
5. $\Phi(S) = \alpha_S S^2/2 + \beta_S S^3/4 - \delta S$
6. $dS/dt = -d\Phi/dS$
7. $d^2B/dt^2 = \gamma_c \delta S - (\omega_B/Q_B) dB/dt - \omega_B^2 B$
8. R_B : a finite-difference audit of (7), never fed back

The control law. $u = \text{clip}(-f_U(x, t) + K_P (\sigma - U), -25, +25)$, where f_U is the drift of U with the command at zero, σ is the target pole and $K_P = 12$. The first term cancels whatever is shoving U (the push); the second pulls U to its pole, harder the farther it is (the pull). The command is held unchanged through every stage of the fourth-order Runge-Kutta step and U is never rewritten afterwards. Inside the authority limit, U converges to its pole at rate K_P ; this is proved (docs/TRACKING_THEOREM.md).

5. The Closed Circle: Why It Cannot Leave Its Bowl

The governing principle is the Unified Circle Principle:

$$\begin{aligned}
 & dX/dt = G(X), \quad X(0) \text{ in } \Omega \\
 & G(X) \cdot n(X) \leq 0 \text{ on the boundary of } \Omega && \text{(the flow points inward at the wall: nothing crosses)} \\
 & \text{grad } L(X) \cdot G(X) \leq 0 && \text{(the bowl's energy } L \text{ only falls)} \\
 & \Rightarrow \lim_{t \rightarrow \infty} X(t) \text{ in } M^* && \text{(every path settles in the bottom of the bowl)}
 \end{aligned}$$

In control engineering the second line is the Nagumo condition for an invariant set and the third is a Lyapunov function. Together they mean: anything that starts inside the container stays inside it, for every disturbance up to a known size, and comes to rest at the bottom. That is a certificate, not a test: it answers every case inside the walls at once.

The engine's states are each closed in this sense: the deviation tank drains faster than it can fill; the alignment's cubic walls push back from far out; the memory forgets; the bath's friction settles it. The rule behind all four is the compass's east-west line: what flows out (drain, damping) must always be able to match what flows in (drive).

The compass. The Omni-Compass rose carries the whole Greek alphabet, Alpha to Omega: the complete set, in a ring whose end runs back into its beginning. Its north-south axis is polarity: Alpha and plus at the top, Omega and minus at the bottom, the two poles of the alignment's double well. Its east-west axis is flow: Beta and the push outward on one side, Gamma and the pull inward on the other, the drive and the damping. The spiral at the center is the attractor every path winds into. The ring closing on itself is the return: when Omni-Compass stops, every lever returns to where it began.

6. The Bowl: Push, Pull and the Two Forces

The bowl (`omnicompass/bowl.py`) is the law that carries the engine's push and pull to every muscle.

Two bands.

- **The cover**, a knob's hard range (the device's or the operator's lowest and highest setting). Every write is clipped to it. Nothing Omni-Compass computes can set a knob outside it.
- **The bowl**, the service reading as a position from 0 (calm) to 1 (the service line). Omni-Compass pulls that position to the bottom of the bowl, the middle (0.5 by default; an operator may set it lower for extra margin). The walls run from 0.05 to 0.95; the last 5% on each side, 10% in all, is cushion.

The force. $F = A \tanh((K_P (p - \text{center}) + K_D v) / A)$, where p is the position, v its rate of change, A the authority.

- **Pull:** $K_P (p - \text{center})$, gentle near the bottom, stronger up the walls.
- **Push:** $K_D v$, the force that meets whatever is shoving the position, and the friction that stops it sloshing. With K_D at or above critical damping the position glides to the middle and stops, with no overshoot and no ringing. Overshoot is wasted energy: force spent going the wrong way and spent again coming back.
- **Smooth, never a hammer:** $\tanh$ bends the force over into its maximum instead of slamming into a wall.
- **Fail up:** past the 0.95 wall the up side goes to its full force at once and the down side may not act until the position is back inside the bowl.

Two forces: antagonist pairs. Like the muscles of an arm, every plug has an up side (adds capacity, power, cooling, speed) and a down side (takes it back), each with its own gain, because adding and taking back do not cost the same: a new machine takes minutes to boot; giving one back is instant. Where a machine has two wires, each side gets its own lever:

Muscle	Up force	Down force
GPU card	clock ceiling raised	power limit lowered (the lid)
Kubernetes	scale out	scale in
Cooling	chiller on, colder setpoint	warmer setpoint, unit released
Battery	discharge	charge, reserve held
Vehicle, joint	motor effort	braking, regeneration

Why the GPU needs both wires. A card's firmware boosts its clock as high as it may while there is work, and its power limiter knocks the clock back each time the draw crosses the limit; on a busy card this happens many times a second, at the top of the clock range where each extra step of speed costs the most watts. With one wire (the power limit) a governor can only move the wall the boost pushes against. With two wires, the clock ceiling sets how high the boost may climb and the power limit becomes a lid that rarely needs to act: the card runs at the bottom of its bowl instead of fighting itself at the top. On a modelled card, the same engine moved from +0.1% work per energy with one wire to +9.0% with two (section 15).

7. The Two-Way Nervous System

Every muscle is wired both ways: a sensory wire in (its meters) and a motor wire out (its knob), with the read-back closing the loop. Between them sits the nervous system (`omnicompass/nervous_system.py`), which decides how much authority each organ has at each moment:

- **Expand is always allowed** (except under a security hold): adding capacity, power, cooling or protection never waits.
- **Contract needs calm and a clean service record:** giving anything back is allowed only while the organ is calm enough and service has been inside its line for the last three decisions; continuous organs give back at most the calm share of their surplus per decision; discrete organs one unit at a time, through a release gate.
- **Fail up:** while service is breached, the knob returns to native at once.
- **Blind means hold:** if a sense goes stale or unreadable, nothing is given back until it returns.

One brain reads every organ at once. Because one law sets every knob, no two muscles fight: when the GPU's watts turn to heat, the cooling knob already knows it is coming; when pods scale up, the power envelope is ready.

PART III - THE HARNESS: PLUGS, WIRES AND THE WIRE CHECK

8. The Plug, the Adapters and the Wire Check

Think of a high-end car stereo: one head unit, one standard plug on its back, and an adapter harness for each make of car that matches the car's factory plug. You never cut a factory wire, and when you pull the stereo, the car works as it did. Omni-Compass is wired the same way.

8.1 The plug (one design for every muscle). Each plug has as many channels as its muscle has knobs. Every channel declares its cover (range), its direction (which way is "more"), its units and how fast it may move. Every plug keeps the same contract (`omnicompass/bowl.py`, `Plug`):

1. **attach** - read the knob once, before any write: the snapshot. It never moves afterwards.
2. **read** - the service reading, into the bowl.
3. **write** - one value, clipped to the cover, then read back from the device.
4. **one writer** - if the knob is found at a value Omni-Compass did not write, someone else owns it: Omni-Compass stops writing and leaves that value alone.
5. **restore** - on stop, the knob returns to the snapshot (where Omni-Compass found it, not the last value it wrote). For muscles whose native controller moves the knob itself, restore hands control back.

8.2 Adapters (one per standard plug). Omni-Compass speaks each industry's standard control interface through an adapter. Status is stated plainly:

Standard	Muscles	Adapter status
Kubernetes API	HPAs, deployments, pods, nodes	built (<code>omni_controller/controller.py</code> , <code>muscles.py</code>)
NVIDIA NVML / <code>nvidia-smi</code>	GPU clock ceiling, power limit	built (<code>omni_controller/gpu_bowl.py</code> , two wires; <code>gpu_governor.py</code> , one wire)
Linux <code>cpufreq</code> , RAPL	CPU frequency ceiling, package watts	built (<code>--cpufreq-policy-root</code> , <code>--rapl-cmd</code>)
Site meter, building controller by command	site watts, supply-air setpoint	built (<code>--power-cmd</code> , <code>--cooling-cmd</code> command templates)
Cloud node groups (Karpenter, Cluster Autoscaler, MachineSet)	machines	built through <code>--node-scale-cmd</code> templates
BACnet, Modbus, SNMP	chillers, air handlers, PDUs, UPS	designed; today reached through the command templates above
OPC UA, EtherNet/IP, PROFINET, EtherCAT, ROS 2, CAN	PLCs, drives, robots, vehicles	designed; modelled in the realm harness
IEC 61850, DNP3, OpenADR, IEEE 2030.5, SunSpec, OCPP	substations, batteries, inverters, chargers	designed; modelled in the realm harness

The engineer's job for one muscle is three things: which adapter, the address of the knob, and its safe range.

8.3 Power-up order. Read-only first; then take the knobs, the least consequential first; on stop, hand them back in reverse order.

8.4 The wire check (mandatory before any write). For the GPU it is one command, `sudo python3 tools/gpu_wire_check.py --gpu 0`, and `scripts/gpu_rented_run.sh` runs it first and stops if it fails:

Step	It proves
1 read	every meter answers: watts, temperature, utilization, limit, clock, top clock
2 snapshot	the start limit is recorded once
3 up wire	with the card busy, a ceiling below its own busy clock pulls the clock down under it; reset, the clock comes back above it
4 down wire	a lower power limit is reported back exactly; the start limit is reported back exactly
5 restore	clocks reset and the start limit read back
6 stop	the governor, stopped by its signal, hands both wires back
7 other writer	a limit set by someone else while the governor runs is left alone, and the governor exits 5

Any failure prints the wire, the step and what the device said, and nothing else runs. That turns a wiring fault from guesswork into one named line to fix.

PART IV - WIRING IT ONTO YOUR STACK, STEP BY STEP

9. Before You Start, and the Eight Levels

9.1 What your system needs

You run	You need
Kubernetes (vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind)	Kubernetes 1.34 or newer (in-place pod resize); metrics-server (<code>kubectl top nodes</code> works); an HPA with a CPU target on each governed service; a readiness probe and a short preStop pause on each service
A response-time feed (strongly recommended at every level)	a CSV per service, <code>elapsed_seconds</code> , <code>latency_ms</code> , <code>ok</code> , written continuously; <code>scripts/latency_probe.py</code> writes one against any HTTP endpoint
NVIDIA GPUs	a driver with <code>nvidia-smi</code> ; root (or the capability to run <code>nvidia-smi -pl</code> and <code>-lgc</code>); on a VM, full GPU passthrough
CPU power control	Linux <code>cpufreq</code> with the <code>schedutil</code> governor; RAPL for package watts; usually bare metal
Site power and cooling	a command that prints site watts, and a command that sets the supply-air setpoint through your building management system

9.2 Get the software and check it

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass.git
cd The-Omni-Compass
pip install -r requirements.txt
python3 verify.py                                # every check; ends with VERIFICATION: PASS
```

Build the container image once:

```
docker build -f deploy/Dockerfile -t <your-registry>/omni-compass:<tag> .
docker push <your-registry>/omni-compass:<tag>
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

9.3 The levels

Take them in order. Each has a pass condition; go to the next level only when it holds. The OFF switch works at every level.

Level 0 - Evaluate without touching anything.

```
python3 verify.py
python3 tools/run_scale.py --runs 10 --scale 1 --out results/scale/eval    # the six organisms, simulated
python3 tools/run_gpu_card.py results/sim/gpu_two_wire/eval              # the two-wire card, modelled
```

Pass: VERIFICATION: PASS. Nothing in your systems is touched.

Level 1 - Watch (read-only).

```
kubectl apply -f deploy/install/omni-compass.yaml    # set the image line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

The install file grants a read-only identity; `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write. Every decision it would take is logged with its reason. Pass: zero writes, and readings your operators agree with.

Level 2 - The pods, on top of your autoscalers. WRITES.

1. Grant patch `horizontalpodautoscalers` and patch `pods/resize` (`deploy/rbac-target.yaml`).

2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.

3. Optional pod muscles, one at a time:

Muscle	Switch	What it does
convey	<code>--latency-file</code> and <code>--cap-deployments ns/name</code>	gives each machine's idle CPU to the serving pods on it
rightsize	<code>--rightsize-deployments ns/name</code>	each pod's CPU request follows its measured use times (1 + headroom), in place
coldstart	<code>--coldstart-deployments ns/name --coldstart-signal ns/configmap</code>	scales a service to zero while no work waits, wakes it when work arrives
batch	<code>--batch</code>	admits held Jobs labelled <code>omnicompass.io/batch=true</code> when there is load and power headroom
batch pace	<code>--batch-pace</code>	pauses Jobs labelled <code>omnicompass.io/pausable=true</code> under power or heat stress, resumes them after
rollout guard	<code>--rollout-guard ns/name</code>	pauses a rollout while change is not permitted
contain	<code>--contain-namespaces ns --contain-cpu-m <m></code>	holds an agent namespace to a CPU budget
security hold	<code>--security-configmap ns/name</code>	key <code>hold: "true"</code> blocks every expansion

Your HPAs keep scaling as before; Omni-Compass sets their targets and raises floors ahead of bursts. Pass: p95, p99 and failed requests no worse than native, over paired runs (section 13).

Level 3 - The machines. WRITES.

- Grant patch nodes and patch pods (deploy/kind/rbac-omni.yaml).
- Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"` and `--node-restore-cmd "<command>"` for the OFF switch.

Your platform	The park/wake command
any cluster, bare metal, kind	<code>bash scripts/kind_nodepool.sh {n}</code>
Karpenter / EKS Auto Mode	the NodePool CPU limit at <code>{n}</code> times node CPU, parked nodes kept
Cluster Autoscaler node group	the group's desired size, scale-down through parking
OpenShift / OKD	the worker MachineSet replicas, parked

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band. Pass: fewer machines in service, with no service gauge worse.

Level 4 - Omni-Compass decides; Kubernetes is the muscle. WRITES. Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; the HPA stays as the fast reflex upward. Pass: as level 3.

Level 5 - A GPU box, two wires. WRITES.

```

sudo python3 tools/gpu_wire_check.py --gpu 0 # must end: WIRED RIGHT
# watch: computes both wires and writes neither
sudo python3 -m omni_controller.gpu_bowl --mode watch --gpus 0 --audit /var/log/omni/gpu.jsonl \
  --latency-file <feed> --slo-ms <p95 target> --floor-w <your lowest watts>
# govern: writes the clock ceiling and the power limit
sudo python3 -m omni_controller.gpu_bowl --mode cap --gpus 0 --audit /var/log/omni/gpu.jsonl \
  --latency-file <feed> --slo-ms <p95 target> --floor-w <your lowest watts>
# OFF: clocks reset, power limit back to the start, read back
sudo touch /tmp/omni-gpu-kill

```

Every `--interval` seconds (default 2) it reads the card's own meters and the response times, places the service in its bowl, and moves the clock ceiling (cover: `--clock-min-share` of the top clock, default 35%, to the top) and the lid (never under `--floor-w`, never over the start limit). A breach, a blind sense or a heat slowdown sends it up at once. The one-wire governor (`omni_controller.gpu_governor`, power limit only) remains available. Pass: work per energy up; requests served and p95 inside the guardrails.

Level 6 - CPU clock and power. WRITES. On bare metal, add to the controller: `--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil --rapl-cmd "<prints CPU package watts>"`. The OFF switch writes every policy's recorded maximum back exactly. Pass: energy per unit of work down, no service gauge worse.

Level 7 - Site power and cooling. WRITES. `--power-cmd "<prints site watts>" --site-limit-w <limit>` brings site power stress into the engine; `--cooling-cmd "<sets {c}>" --cooling-min-c 18 --cooling-max-c 27 --cooling-restore-c 22` lets it hold the supply-air setpoint in its band. CPU and GPU sharing one power budget (`hardware/node_exchange.py`) and GPU groups sharing a site budget (`hardware/site_exchange.py`) run in simulation today. Batteries are designed as an organ, not yet wired.

10. Stack by Stack

Stack	Levels	Notes
Vanilla Kubernetes, kind, Rancher	1-7	as written
Amazon EKS	1-5	machines via Cluster Autoscaler node group or Karpenter; CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances
Google GKE, Azure AKS	1-5	the provider's node-pool size as the park/wake command
Red Hat OpenShift / OKD	1-5	the same permissions through a Role; machines via the worker MachineSet
NVIDIA GPU servers without Kubernetes	5	the two-wire GPU governor alone
Bare-metal CPU servers	6	cpufreq and RAPL through sysfs
Slurm / HPC	5 on the GPU nodes	a job-level Slurm connector is not built
Building management	7	through your BMS's command line or API, in the command templates

PART V - OPERATING IT

11. The OFF Switch, the Rules, and the Log

The rules Omni-Compass keeps on your system.

1. One OFF switch, in a human hand: the kill file (or `OMNI_KILL=1`, or `SIGTERM` to the GPU governor) returns every setting to its recorded original, reads each back, and stops all action.
2. It records before it acts (annotations on Kubernetes objects; the snapshot line in the GPU audit).
3. It watches before it writes.
4. It never acts blind.
5. Service first: while response time is over its target, and for `--slow-clear` decisions after, nothing is given back.
6. Every change is read back; no new order goes on top of one that has not landed.
7. The living band and the cover: no organ is driven outside its range; machines are parked, never switched off by Omni-Compass; it never evicts or moves a pod.
8. One writer: if anyone else changes a knob, Omni-Compass stops writing it and leaves it alone.
9. Everything is logged: every read, decision, write and reason goes to the audit log.

Reading the log.

Line	Meaning
gate: a sense is blind	a reading failed; nothing is given back until it returns
gate: pods scaling up	pods first, machines after
decision failed (n in a row)	the cluster could not be reached; nothing was written; turn it OFF for native
decided_by: bowl	the bowl set the wires this decision
decided_by: fail_up / blind_fail_up	the service crossed the 0.95 wall, or a sense went blind: full capacity at once
decided_by: thermal_hold	the card reported a heat slowdown; nothing was tightened
foreign_writer	someone else changed a knob; Omni-Compass now observes only
restored ... ok: true	the OFF switch put every setting back and read it back

12. Maintenance, Upgrades and Security

- Run `python3 verify.py` after every upgrade; it must end `VERIFICATION: PASS`.
- Upgrade in watch mode first; take the levels again from level 1.
- The container runs non-root, read-only, with no capabilities; the Kubernetes identity has only the permissions of the level you run.
- The GPU governor needs root only for `nvidia-smi -pl` and `-lgc`.
- Report a security issue as `SECURITY.md` describes.

PART VI - PROVING IT

13. Paired Runs and Receipts on Your Own System

1. **Paired runs.** Run your service the same way twice, once native and once with Omni-Compass on top, back to back on the same machines, the order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this on Kubernetes; `scripts/gpu_rented_run.sh` does it on a GPU box, in one command: wire check, smoke, the six organisms with the card inside, then the preregistered confirmation.
2. **The switch drill** after every run: turn Omni-Compass OFF; confirm every setting is back at its recorded original; turn it ON.
3. **The receipt.** Each gauge: native, Omni-Compass, the change, the 95% interval of the difference, and whether the interval excludes zero. A change is proven only when it does.
4. **The label, by rule written before the run:** SUPERIOR WITHIN GUARDRAILS / ENERGY IMPROVEMENT WITH SERVICE TRADEOFF / NONINFERIOR / NOT ESTABLISHED / WORSE / INVALID. Guardrails: work not lower by more than 1%; the share of time outside the service line not higher by more than 1 percentage point. The band-first rule is stricter: no win is claimed while the time outside the service line is above native's.

At scale (simulated). The six organisms run on GitHub's machines (Actions, workflow `six`) or on any machine (`bash scripts/scale_ladder.sh`) at 1, 10, 100 and 1,000 paired runs and at 1, 10, 100 and 1,000 copies of each organism on one clock. Real Kubernetes runs on GitHub's machines (workflow `benchmark-reps`).

14. Evidence Classes and How to Read a Result

Class	Rung	What it is	What it can show
T / V	E1	deterministic tests, proofs, Python against the C++ twin	the law is what it says, and both languages agree
S	E2	simulation on a modelled plant	whether the law helps the model, and where it breaks
L	E3	real software (Kubernetes on kind), no hardware meter	real decisions on real software; energy there is a declared model
P	E4	a physical meter (the GPU's own power reading)	the hardware's own answer

Read every number with its class beside it. A simulation number is never quoted as a hardware result. When a receipt's energy line is modelled, the receipt says so.

15. Results to Date

Result	Class	Source
Real Kubernetes, set 24 (10 paired runs): machines in service -31.6%, p95 response -60.1%, p99 -64.1%, HPA replicas -38.6%, failed requests 0 on both, total CPU including Omni-Compass's own -1.8% (not significant)	L	GitHub run 36983865216
Real Kubernetes, set 23 (10 paired runs): p95 -62.2%, replicas -36.6%, machines in service -28.7%, failed requests 0	L	<code>results/live/LIVE_REPS_23.md</code>
Modelled GPU card, fresh seeds: two-wire bowl +9.0% work per energy (energy -8.2%), one-wire governor +0.1%, both wires restored every run	S	<code>results/sim/gpu_two_wire/</code>

Result	Class	Source
Six organisms, 1,000 runs each at 1x: work per energy +0.30% (compute), +0.23% (physics), +0.21% (energy), +0.25% (distribution), +0.21% (four stacked), +0.22% (whole tower); every knob handed back; time outside the service line about +0.2 points above native in each, so the band-first rule is not yet met	S	GitHub workflow <code>six</code>
Real GPU (NVIDIA A10) on the two-wire engine	P	in progress; results arrive as <code>results/gpu/omni-gpu-<stamp>.tar.gz</code>

PART VII - CODE, TWINS AND TERMS

16. License and Commercial Terms

The software and this manual are licensed under the Omni-Compass Evaluation License (LICENSE): evaluation and simulation use only. Everything else - commercial use, production use, operating any system beyond evaluation, redistribution, a hosted or managed service, incorporation into a product or service, or using the software or its results to build a competing product - requires a written Omni-Compass Enterprise License signed by The Omni-Compass LLC and paid for. No patent or trademark license is granted for any other use. Contributions are accepted only on the terms in CONTRIBUTING.md, which assign their rights to The Omni-Compass LLC.

17. Python, C++ and the Seal

The laws are twinned: each has a Python version and a C++20 version that give the same answers, proven by a parity test on every build (`cmake -S cpp -B cpp/build && cmake --build cpp/build`).

Law	Python	C++	Proven by
core engine	omniconpass/core.py	cpp/src/core.cpp	500 frozen fixtures
governor	omniconpass/adapter.py	cpp/src/governor.cpp	tests/test_cpp_governor_parity.py
safety shield	omniconpass/shield.py	cpp/src/shield.cpp	tests/test_cpp_shield_parity.py
HPA replica law	fleet/harness.py	cpp/src/hpa.cpp	tests/test_cpp_hpa_parity.py
closure law	omniconpass/closure.py	cpp/src/closure.cpp	tests/test_cpp_closure_parity.py
conveyance law	omniconpass/conveyance.py	cpp/src/conveyance.cpp	tests/test_cpp_conveyance_parity.py
nervous system	omniconpass/nervous_system.py	cpp/src/nervous_system.cpp	tests/test_cpp_twins_parity.py
compass and ledger	omniconpass/compass.py, storage.py	cpp/src/compass.cpp	tests/test_cpp_twins_parity.py
GPU governor rules	omni_controller/gpu_governor.py	cpp/src/gpu_rules.cpp	tests/test_cpp_twins_parity.py

The seal (`results/SEAL.json`) holds the SHA-256 fingerprint of every twinned file, written only after every parity test passes. `verify.py` fails, naming the file, if any sealed file changes afterwards. The bowl law (`omniconpass/bowl.py`) and the two-wire GPU governor are in Python today; their C++ twins are next.

BACK MATTER

Glossary

Term	Meaning
Antagonist pair	the up side and the down side of a muscle's control, each with its own gain
Authority	how far the nervous system lets an organ move this decision
Band	a reading's or a knob's safe range
Bowl	the band seen as a position from 0 (calm) to 1 (the line), with its bottom in the middle
Cover	a knob's hard range; every write is clipped to it
Cushion	the 5% at each edge of the bowl, 10% in all
Fail up	the return to full capacity the moment service crosses the wall or a sense goes blind
Governor	the process that reads, decides and writes; Omni-Compass
Muscle	any machine, service or controller Omni-Compass reads and sets
Native	the system as it runs without Omni-Compass
Organism	a set of muscles run together on one clock
Plug	the two-way connection to one muscle: read, write, read back, restore
Receipt	the paired record of native against Omni-Compass for one run
Snapshot	a knob's value read once before the first write; the restore point
Wire check	the test that proves every wire follows, reads back and returns before anything runs
Work per energy	work done divided by energy used; the primary outcome

Appendix A - Command Reference

Task	Command	
Verify everything	<code>python3 verify.py</code>	
Six organisms, simulated	<code>python3 tools/run_scale.py --runs N --scale K --out <dir></code>	
The full ladder	<code>bash scripts/scale_ladder.sh</code>	
Two-wire card, modelled	<code>python3 tools/run_gpu_card.py <dir> [fresh]</code>	
GPU wire check	<code>sudo python3 tools/gpu_wire_check.py --gpu 0</code>	
GPU, whole benchmark in one command	<code>sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &</code>	
The six organisms with the real card inside	<code>python3 tools/run_hil.py --out <dir></code>	
GPU governor, two wires	<code>`sudo python3 -m omni_controller.gpu_bowl --mode watch</code>	cap ...`
Kubernetes, watch	<code>kubectl apply -f deploy/install/omni-compass.yaml</code>	

Task	Command	
Kubernetes paired runs	<code>scripts/kind_paired.sh</code> , then <code>tools/live_reps.py</code>	
OFF (Kubernetes)	<code>touch /tmp/omni.kill</code>	
OFF (GPU)	<code>sudo touch /tmp/omni-gpu-kill</code>	
Seal check	<code>python3 tools/seal.py --check</code>	

Appendix B - File Map

Path	Contents
<code>omnicompass/</code>	the engine, governor, nervous system, shield, compass, conveyance law, the bowl
<code>omni_controller/</code>	the Kubernetes controller and muscles; the GPU governors (one and two wires)
<code>realms/</code>	the 656-muscle catalog, the plant models, the realm harness, the bowl on every muscle, the modelled card
<code>tools/</code>	benchmarks, receipts, the wire check, the scale ladder, manifests, seals
<code>scripts/</code>	one-command runs (GPU, Kubernetes, ladder)
<code>deploy/</code>	container image, install and permission files
<code>cpp/</code>	the C++20 twins
<code>results/</code>	every published result, with its raw files and checksums
<code>docs/</code>	this manual, the preregistrations, the evidence ledger, the theorem, the realm study
<code>LICENSE, NOTICE, LICENSES/</code>	the license and notices

Appendix C - The Equations in Full

```

(1) dE/dt    = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt    = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,    |u| <= U_AUTHORITY = 25
(3) dI_U/dt  = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff    = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S)   = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt    = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt    = B_dot ; dB_dot/dt = gamma_c delta S - (omega_B / Q_B) B_dot - omega_B^2 B
(8) R_B[n]   = finite-difference audit of (7); never fed back into the state

```

Control (one micro step, zero-order hold across all four RK4 stages):

```
u = clip(-f_U(x, t) + K_P (sigma - U), -U_AUTHORITY, +U_AUTHORITY), K_P = 12, sigma in {-1, +1}
```

The bowl (every muscle):

```

p = position of the service reading in its band (0 calm, 1 the line)
F = A tanh((K_P (p - center) + K_D v) / A),    K_D >= critical damping
p >= 0.95 => full up force; down side held
knob <- clip(knob + g_side F span, cover)

```

Parameter ranges, defaults and the proof of convergence: `omnicompass/core.py`, `docs/TRACKING_THEOREM.md`, `docs/CANONICAL_ENGINE.md`.

Appendix D - Metrics

Every gauge, where it comes from, and whether it is measured or modelled: `docs/METRICS_CATALOG.md`.

Appendix E - Troubleshooting

Symptom	Cause	Fix
Wire check FAIL 3 up wire (lock)	the driver or VM refuses clock locking	use bare metal or full passthrough; or run the one-wire governor
Wire check FAIL 3 up wire (follows down)	the card ignores the ceiling	update the driver; check for another management agent holding clocks
Wire check FAIL 4 down wire	the power limit is refused or capped by the board	check <code>nvidia-smi -q -d POWER</code> ; run as root
another copy of this test is already running	a second copy was started	wait for the first or reboot; start once
something else is using the GPU	another process holds the card	stop it; the benchmark must run alone
Governor exit 5	another writer changed a knob	find the other controller; Omni-Compass left its value alone
Governor exit 3	a restore did not read back	restore by hand (<code>nvidia-smi -rgc, -pl <start></code>); investigate before rerunning
decision failed (n in a row)	the cluster API is unreachable	turn it OFF; native runs on

Appendix F - Evidence Map

docs/EVIDENCE_LEDGER.md (every claim and its class), docs/CLAIMS_REGISTER.md (what is claimed and what is not), docs/GPU_PREREGISTRATION.md and docs/REALMS_PREREGISTRATION.md (the rules written before each run), STATE_OF_PLAY.md (where everything stands), HANDOFF.md (every command in one page).

Contact

Licensing, pilots and the Omni-Compass Enterprise License: **The Omni-Compass LLC**.

Copyright (c) 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.